POWERMANAGER

Especificação Consolidada de Regras de Negócio e Arquitetura

Versão documental: 1.1
Data de consolidação: 10/11/2026
Base: Especificação Consolidada versão 1.0 (09/11/2026), decisões posteriores aprovadas nas conversas do projeto e inspeção do código disponibilizado em 10/11/2026.
Status: regras de negócio consolidadas; refatoração autorizada e em andamento.

Controle de leitura e autoridade das informações

Este documento separa quatro categorias para evitar que uma decisão, uma proposta ou uma estrutura de código seja confundida com outra:

1. **Regra aprovada:** decisão de negócio confirmada no projeto.
2. **Implementação verificada:** elemento encontrado no código-fonte inspecionado.
3. **Pendente:** ponto que ainda exige implementação, teste ou validação regulatória.
4. **Observação técnica:** recomendação ou consequência técnica que não altera, por si só, uma regra aprovada.

A especificação versão 1.0 é a base original. As decisões posteriores registradas neste documento complementam ou detalham aquela versão. Quando houver diferença entre a especificação original e uma decisão posterior expressamente aprovada, prevalece a decisão posterior.

A presença de uma entidade JPA, anotação, DTO ou método não comprova que todo o fluxo de negócio esteja concluído. O status de implementação deve ser avaliado separadamente.

1. Objetivo do projeto

O PowerManager é um sistema de gerenciamento de consumo e faturamento de energia elétrica, inicialmente direcionado aos consumidores do **Grupo B no Brasil**.

O sistema deverá permitir:

- cadastro e gerenciamento de clientes;
- cadastro de unidades consumidoras;
- gerenciamento de medidores;
- registro e histórico de leituras;
- administração de tarifas de energia elétrica;
- gerenciamento de bandeiras tarifárias;
- configuração dos tributos aplicáveis;
- geração e gerenciamento de faturas;
- consulta de consumo e histórico de faturamento;
- gerenciamento de usuários e permissões.

O objetivo não é simplesmente calcular o consumo multiplicando kWh por um valor fixo. O sistema deverá representar os componentes que formam uma fatura de energia elétrica, respeitando suas regras próprias de aplicação, localização e vigência.

A arquitetura deverá permitir futuras expansões sem exigir uma remodelagem completa do domínio.

2. Premissas gerais

1. O sistema será inicialmente direcionado ao mercado brasileiro.
2. O escopo inicial será o Grupo B.
3. A modalidade tarifária inicial será a convencional. A modalidade branca fica prevista para evolução futura.
4. Valores monetários, consumo e tarifas serão representados com `BigDecimal`.
5. Datas de vigência serão representadas com `LocalDate`.
6. Datas e horários de registros serão representados com `LocalDateTime` ou tipo temporal equivalente.
7. Identificadores das entidades serão UUID.
8. A arquitetura manterá separação entre entidades, DTOs, mappers, repositories, services e controllers.
9. Regras de negócio permanecerão na camada de serviço ou em componentes de domínio apropriados.
10. Valores históricos de faturamento não poderão ser alterados retroativamente por mudanças posteriores nas tarifas, bandeiras ou regras tributárias.
11. O sistema possuirá autenticação e autorização.
12. Operações administrativas serão restritas a usuários autorizados.
13. O sistema possuirá testes unitários e de integração.

14. A evolução do banco de dados será controlada por migrations versionadas.

3. Perfis de usuário

3.1 ADMINISTRADOR

O administrador poderá:

- cadastrar e gerenciar clientes;
- cadastrar e gerenciar unidades consumidoras;
- administrar medidores e leituras;
- cadastrar e atualizar tarifas;
- cadastrar valores de bandeiras e períodos oficiais de acionamento;
- cadastrar regras tributárias;
- consultar e administrar faturas;
- gerenciar usuários;
- realizar operações administrativas.

3.2 CLIENTE

O cliente poderá:

- consultar seus próprios dados;
- consultar suas unidades consumidoras;
- consultar medidores e informações de consumo vinculados às suas unidades, conforme permissões;
- consultar suas faturas e histórico.

Um cliente não poderá acessar dados de outro cliente apenas informando o identificador de um recurso.

3.3 Enum `TipoUsuario`

Valores:

- `ADMINISTRADOR`
- `CLIENTE`

3.4 Associação entre usuário e cliente

- Um usuário do tipo `CLIENTE` deverá estar associado a um `Cliente`.
- Um usuário do tipo `ADMINISTRADOR` não deverá possuir associação com `Cliente`.
- Um `Cliente` poderá existir sem possuir conta de acesso.
- Um cliente terá no máximo um usuário associado.
- A exclusão de um usuário não deverá excluir o cliente.
- A coerência entre `tipoUsuario` e a presença/ausência da associação será validada no service, além das constraints estruturais aplicáveis.

4. Modelagem do domínio

4.1 Cliente

Representa a pessoa física ou jurídica que utiliza o sistema.

Responsabilidades

- armazenar os dados cadastrais do consumidor;
- possuir relacionamento com uma ou mais unidades consumidoras;
- ser associado ao usuário de perfil `CLIENTE`, quando aplicável.

A entidade `Cliente` não deverá concentrar os dados de todas as instalações do consumidor. O endereço da instalação pertence à `UnidadeConsumidora`.

Campos definidos

Campo	Tipo	Observação
`id`	UUID	Identificador
`nome`	String	Nome do cliente
`email`	String	E-mail

`telefone`	String	Telefone

Relacionamentos

- Um `Cliente` poderá possuir várias `UnidadeConsumidora`.
- Uma `UnidadeConsumidora` pertencerá a um único `Cliente`.
- `Cliente 1:N UnidadeConsumidora`.

Endereço, cidade e estado foram retirados de `Cliente` e transferidos para `UnidadeConsumidora`.

4.2 UnidadeConsumidora

Representa a instalação consumidora de energia elétrica. É uma das entidades centrais do sistema.

Uma pessoa poderá possuir diversas unidades consumidoras, inclusive em endereços diferentes.

Responsabilidades

- identificar a instalação;
- armazenar informações de localização;
- determinar características tarifárias do consumidor;
- relacionar medidores;
- relacionar faturas;
- fornecer contexto para aplicação de tarifas e tributos.

Campos definidos

Campo	Tipo	Observação
`id`	UUID	Identificador
`codigo`	String	Código da unidade consumidora
`endereco`	String	Endereço da instalação
`cidade`	String	Município
`estado`	String	UF
`cep`	String	CEP
`distribuidora`	String	Distribuidora responsável
`classeConsumo`	Enum	Classe de consumo
`subgrupoTarifario`	Enum	Subgrupo tarifário
`modalidadeTarifaria`	Enum	Modalidade
`ativa`	Boolean	Situação da unidade
`cliente`	Cliente	Titular obrigatório

Relacionamentos

- Uma unidade pertence a um `Cliente`.
- Uma unidade poderá possuir vários `Medidor` ao longo do tempo.
- Uma unidade poderá possuir várias `Fatura`.
- `Cliente 1:N UnidadeConsumidora`.
- `UnidadeConsumidora 1:N Medidor`.
- `UnidadeConsumidora 1:N Fatura`.

Regras

- O código da unidade deverá ser único dentro da distribuidora.
- A restrição de unicidade será composta por `(distribuidora, codigo)`.
- A unidade pertence obrigatoriamente a um cliente.
- A unidade poderá ser desativada; não se deverá apagar fisicamente o histórico de faturamento.
- A distribuidora permanecerá como `String` inicialmente. Evolução para entidade própria fica futura.
- Código IBGE poderá ser incorporado se necessário para precisão tributária municipal.

Mapeamento definido

- `Cliente.unidadeConsumidoras`: `@OneToMany(mappedBy = "cliente", fetch = LAZY)`.
- `UnidadeConsumidora.cliente`: `@ManyToOne(fetch = LAZY, optional = false)`.
- Não utilizar cascade de remoção nem `orphanRemoval` para apagar histórico por exclusão do relacionamento.

4.3 Medidor

Representa o equipamento responsável por registrar o consumo de energia elétrica.

Campos mantidos

- `id` : UUID.
- `numero` : String.
- `tipoMedidor` : enum.
- `tipoLigacao` : enum.

Campos removidos ou transferidos

Campo anterior	Decisão
`cliente`	Substituído por `unidadeConsumidora`
`classeConsumo`	Transferido para `UnidadeConsumidora`
`dias`	Removido
`localizacao`	Removido; endereço pertence à `UnidadeConsumidora`

Novos campos

Campo	Tipo	Descrição
`ativo`	Boolean	Indica se está em operação; padrão `true`
`dataInstalacao`	LocalDate	Data da instalação
`dataRetirada`	LocalDate nullable	Data da retirada, quando aplicável

Relacionamentos

- Um `Medidor` pertence a uma `UnidadeConsumidora` .
- Um `Medidor` possui várias `Leitura` .
- Uma unidade poderá manter histórico de vários medidores.
- Normalmente, uma unidade terá apenas um medidor ativo por vez.

Regras de negócio

1. Um medidor deverá possuir identificação única.
2. Um medidor deverá estar associado a uma unidade consumidora.
3. Uma unidade poderá possuir histórico de vários medidores.
4. Normalmente, uma unidade terá apenas um medidor ativo por vez.
5. A substituição de medidor deverá preservar o histórico.
6. O tipo de medidor não determina diretamente a tarifa.
7. O tipo de ligação poderá ser utilizado em verificações relacionadas às condições de fornecimento e consumo mínimo.
8. O consumo mínimo não deverá ser aplicado indiscriminadamente; dependerá das condições regulatórias e técnicas pertinentes.
9. `dataRetirada` , quando informada, deverá ser posterior à instalação.
10. A coerência entre `ativo` e `dataRetirada` será validada no service.

Identificação

O número do medidor continuará globalmente único conforme o modelo definido para o projeto.

> **Decisão de escopo:** a modelagem de Medidor foi revisada e aprovada. Não reabrir suas decisões já consolidadas sem solicitação expressa de revisão.

4.4 Leitura

Representa cada registro de medição realizado em determinado medidor.

Campos definidos

Campo	Tipo	Descrição
`id`	UUID	Identificador
`medidor`	Medidor	Equipamento associado
`valorLeitura`	BigDecimal	Valor acumulado registrado
`dataLeitura`	LocalDateTime	Momento da leitura
`tipoLeitura`	Enum	`REAL` ou `ESTIMADA`

Precisão definida para `valorLeitura`: `DECIMAL(12,3)`.

Regras de negócio

1. Toda leitura deverá pertencer a um medidor.
2. O valor deverá ser não negativo.
3. A leitura deverá possuir data e horário.
4. As leituras deverão respeitar a sequência cronológica.
5. O consumo será calculado pela diferença entre a leitura atual e a leitura anterior.
6. Uma leitura posterior não poderá apresentar valor inferior à anterior sem justificativa operacional válida.
7. Leituras duplicadas deverão ser tratadas.
8. A substituição de medidor deverá preservar o histórico anterior.

Fórmula

```
Consumo = LeituraAtual - LeituraAnterior
```

Exemplo:

- leitura anterior: 1.250 kWh;
- leitura atual: 1.400 kWh;
- consumo: 150 kWh.

Pontos de implementação

O service deverá tratar rollover/reinicialização, troca de medidor e leituras estimadas. Não assumir que uma redução no valor acumulado seja consumo negativo sem antes tratar as situações operacionais pertinentes.

5. Modelagem tarifária

5.1 Taxa

A entidade `Taxa` representa um registro de tarifa de energia elétrica, composto por TE e TUSD, com vigência e enquadramento tarifário.

Os dados deverão refletir informações homologadas/divulgadas pelas autoridades e distribuidoras competentes. Inicialmente, o cadastro será manual e administrativo.

Campos definidos

Campo	Tipo	Descrição
`id`	UUID	Identificador
`distribuidora`	String	Distribuidora
`subgrupoTarifario`	Enum	Subgrupo
`classeConsumo`	Enum	Classe
`modalidadeTarifaria`	Enum	Modalidade
`tarifaEnergia`	BigDecimal	TE
`tarifaUsoDistribuicao`	BigDecimal	TUSD
`inicioVigencia`	LocalDate	Início
`fimVigencia`	LocalDate nullable	Fim

Precisão

TE e TUSD utilizarão `DECIMAL(12,6)`.

Componentes

- **TE — Tarifa de Energia:** componente associado à energia elétrica consumida.
- **TUSD — Tarifa de Uso do Sistema de Distribuição:** componente associado à utilização do sistema de distribuição.

Fórmula tarifária básica

```
ValorEnergia = Consumo × (TE + TUSD)
```

Exemplo:

- consumo: 150 kWh;
- TE: R$ 0,40/kWh;

- TUSD: R$ 0,45/kWh;
- resultado: `150 × (0,40 + 0,45) = R$ 127,50`.

Esse resultado representa somente a composição tarifária básica, não necessariamente o total da fatura.

Regras de negócio

1. TE não poderá ser negativa.
2. TUSD não poderá ser negativa.
3. Valores iguais a zero serão permitidos.
4. A data inicial será obrigatória.
5. A data final poderá ser nula.
6. A data final deverá ser posterior à inicial.
7. Não poderão existir períodos sobrepostos para a mesma combinação de distribuidora, subgrupo, classe e modalidade.
8. Tarifas encerradas não poderão ser reabertas por atualização comum.
9. Um reajuste deverá encerrar a tarifa anterior e criar um novo registro.
10. A tarifa aplicada será selecionada conforme a data de competência da fatura.
11. A tarifa deverá corresponder às características tarifárias da unidade consumidora.

Vigência

A seleção da tarifa deverá considerar o início da competência. A prevenção de sobreposição deverá ser garantida pela lógica de serviço, de forma transacional.

Campos antigos removidos

Não deverão permanecer como componentes genéricos de `Taxa`:

- `custoPorUnidade`;
- `aluguelMedidor`;
- `taxaServico`;
- `impostoServico`;
- `cess`;
- `taxaFixa`.

Qualquer componente adicional deverá possuir modelagem e regra de aplicação próprias.

5.2 SubgrupoTarifario

Enum:

- `B1`
- `B2`
- `B3`
- `B4`

O escopo inicial será o Grupo B. As combinações permitidas entre subgrupo e classe deverão respeitar a classificação regulatória. Não assumir que todas as combinações são válidas.

5.3 ModalidadeTarifaria

Enum:

- `CONVENCIONAL`
- `BRANCA`

A modalidade convencional será implementada inicialmente. A modalidade branca ficará prevista para evolução futura, pois exige tratamento de períodos tarifários e consumo por faixa horária.

5.4 ClasseConsumo

Manter o enum existente após conferência de compatibilidade com a classificação regulatória. O primeiro cenário de testes utilizará a classe residencial.

6. Bandeiras tarifárias

As bandeiras são independentes da entidade `Taxa`. Não deverão ser armazenadas como campo fixo da tarifa.

6.1 BandeiraTarifaria

Armazena o valor adicional oficial por kWh para cada tipo de bandeira e sua vigência.

Campos

Campo	Tipo	Descrição
`id`	UUID	Identificador
`tipoBandeira`	Enum	Tipo
`valorAdicional`	BigDecimal	Acréscimo por kWh
`inicioVigencia`	LocalDate	Início
`fimVigencia`	LocalDate nullable	Fim

Enum `TipoBandeira`

- `VERDE`
- `AMARELA`
- `VERMELHA_PATAMAR_1`
- `VERMELHA_PATAMAR_2`

Regras

1. O valor adicional deverá ser não negativo.
2. O registro deverá possuir período de vigência.
3. O registro deverá representar os valores oficiais aplicáveis.
4. O administrador poderá cadastrar e atualizar os dados.
5. O sistema deverá preservar os valores históricos utilizados nas faturas.

6.2 AcionamentoBandeira

O cadastro do valor adicional e a definição da bandeira oficialmente acionada são responsabilidades diferentes.

`AcionamentoBandeira` registra:

- tipo de bandeira;
- início e fim da vigência;
- data de divulgação;
- referência oficial.

A bandeira aplicada deverá corresponder àquela oficialmente divulgada para o período.

6.3 FaturaBandeiraAplicada

Registra a aplicação da bandeira dentro de uma fatura, preservando os dados utilizados no cálculo.

Campos

- `id`;
- `fatura`;
- `acionamentoBandeira`;
- `tipoBandeira` (snapshot);
- `inicioAplicacao`;
- `fimAplicacao`;
- `valorAdicionalKwh` (snapshot);
- `consumoConsideradoKwh`;
- `valorAdicional`.

Precisões definidas no modelo:

- `valorAdicionalKwh`: `DECIMAL(12,6)`;
- `consumoConsideradoKwh`: `DECIMAL(12,3)`;
- `valorAdicional`: `DECIMAL(14,2)`.

Aplicação no ciclo de faturamento

O ciclo de faturamento poderá atravessar mais de um período de bandeira. A aplicação deverá considerar os dias/períodos efetivamente abrangidos, em conformidade com a regra regulatória aplicável.

Não presumir uma bandeira única para todo o ciclo. Não presumir rateio proporcional simples do consumo sem validação regulatória.

7. Tributação

Os tributos não deverão permanecer dentro da entidade `Taxa`. Cada tributo possui regras, bases de cálculo, competências, localização e abrangência próprias.

7.1 Tributos considerados

- **ICMS:** estadual.
- **PIS:** federal.
- **COFINS:** federal.
- **CIP/COSIP:** municipal.

7.2 RegraTributaria

Campos

Campo	Tipo	Descrição
`id`	UUID	Identificador
`tipoTributo`	Enum	Tributo
`abrangencia`	Enum	Federal, estadual ou municipal
`tipoCalculo`	Enum	Percentual ou valor fixo
`estado`	String	UF, quando aplicável
`municipio`	String	Município, quando aplicável
`codigoIbge`	String	Identificador municipal, quando aplicável
`aliquota`	BigDecimal nullable	Alíquota
`valorFixo`	BigDecimal nullable	Valor fixo
`inicioVigencia`	LocalDate	Início
`fimVigencia`	LocalDate nullable	Fim

Enums

`TipoTributo`:

- `ICMS`
- `PIS`
- `COFINS`
- `CIP_COSIP`

`AbrangenciaTributaria`:

- `FEDERAL`
- `ESTADUAL`
- `MUNICIPAL`

`TipoCalculoTributario`:

- `PERCENTUAL`
- `VALOR_FIXO`

7.3 FaturaTributo

Armazena o tributo efetivamente aplicado à fatura e os parâmetros utilizados como snapshot.

Campos

- `id`;
- `fatura`;
- `regraTributaria`;
- `tipoTributo` (snapshot);
- `tipoCalculo` (snapshot);
- `aliquotaAplicada`;
- `baseCalculo`;
- `valorFixoAplicado`;
- `valorTributo`.

Precisões definidas:

- alíquota: `DECIMAL(12,6)`;
- base de cálculo: `DECIMAL(14,2)`;

- valor fixo aplicado: `DECIMAL(14,2)`;
- valor do tributo: `DECIMAL(14,2)`.

7.4 Regras e limites

1. Os tributos serão configurados separadamente.
2. A abrangência e a localização deverão ser coerentes com o tributo.
3. Alíquota e valor fixo não poderão ser negativos.
4. A regra deverá possuir início de vigência; o fim poderá ser nulo.
5. O fim, quando informado, deverá ser posterior ao início.
6. A fatura deverá preservar os parâmetros efetivamente aplicados.
7. A coerência entre `tipoCalculo` e os campos preenchidos será validada no service.

7.5 Pontos que não estão fechados

As fórmulas, bases de cálculo, hipóteses de incidência, ordem de cálculo e arredondamentos específicos de ICMS, PIS, COFINS e CIP/COSIP deverão ser validados conforme a legislação aplicável antes de implementar o cálculo definitivo.

Não presumir que todos os tributos incidem sobre a mesma base ou seguem a mesma metodologia.

8. Fatura

8.1 Relacionamento

A fatura pertence à `UnidadeConsumidora`, e não diretamente ao `Medidor`.

Uma unidade consumidora poderá possuir várias faturas ao longo do tempo.

8.2 Campos definidos

Campo	Tipo	Descrição
`id`	UUID	Identificador
`unidadeConsumidora`	UnidadeConsumidora	Unidade associada
`competencia`	LocalDate	Primeiro dia do mês de referência
`inicioPeriodo`	LocalDate	Início do ciclo
`fimPeriodo`	LocalDate	Fim do ciclo
`consumoKwh`	BigDecimal	Consumo faturado
`origemConsumo`	Enum	`LEITURA` ou `MANUAL`
`tarifaEnergiaTE`	BigDecimal	TE aplicada
`tarifaUsoDistribuicaoTUSD`	BigDecimal	TUSD aplicada
`valorTE`	BigDecimal	Valor calculado de TE
`valorTUSD`	BigDecimal	Valor calculado de TUSD
`valorBandeira`	BigDecimal	Total adicional de bandeiras
`valorSubtotal`	BigDecimal	Subtotal antes dos tributos
`valorTotal`	BigDecimal	Total após componentes aplicáveis
`dataEmissao`	LocalDate	Emissão
`dataVencimento`	LocalDate	Vencimento
`dataPagamento`	LocalDate nullable	Pagamento
`status`	Enum	Situação

Precisões:

- consumo: `DECIMAL(12,3)`;
- TE/TUSD unitárias: `DECIMAL(12,6)`;
- valores monetários: `DECIMAL(14,2)`.

8.3 Unicidade

Não poderão existir duas faturas para a mesma combinação:

`UnidadeConsumidora + Competencia`

A restrição deverá existir tanto na aplicação quanto no banco.

8.4 Competência

A competência será representada pelo primeiro dia do mês de referência.

Exemplo: novembro de 2026 = `2026-11-01`.

A seleção da tarifa deverá considerar a tarifa válida no início da competência.

8.5 Snapshot dos valores

A fatura deverá armazenar os valores utilizados no momento de sua geração. Alterações posteriores em tarifa, bandeira ou regra tributária não poderão modificar os valores históricos da fatura.

O snapshot deverá permitir identificar os componentes tarifários, valores de bandeira, parâmetros tributários, bases, alíquotas e valores efetivamente utilizados.

8.6 Origem do consumo

A geração de faturas deverá permitir:

1. consumo calculado a partir de leituras;
2. consumo informado manualmente, especialmente para simulações.

A origem deverá ser identificada.

8.7 Status e pagamento

Manter inicialmente:

- `PENDENTE`
- `PAGO`

Regras:

1. Uma fatura recém-gerada será `PENDENTE`.
2. Após o registro do pagamento, passará para `PAGO`.
3. Não será permitido registrar pagamento duplicado.
4. O pagamento deverá possuir data registrada.
5. A operação deverá ser transacional.
6. O pagamento será inicialmente registrado manualmente.
7. Não haverá integração com gateway financeiro nesta primeira versão.

8.8 Composição do subtotal e total

- `valorSubtotal` = `valorTE + valorTUSD + valorBandeira`.
- `valorTotal` = subtotal acrescido dos tributos e demais componentes aplicáveis, sem dupla contagem.

A forma exata de composição de tributos dependerá das regras regulatórias validadas.

8.9 Precisão e arredondamento

- Utilizar `BigDecimal`.
- Valores monetários finais terão duas casas decimais.
- Arredondamento geral inicial: `HALF_UP`.
- Valores unitários deverão preservar precisão suficiente durante os cálculos.
- Não utilizar `double` ou `float`.
- A regra de arredondamento específica de cada componente deverá ser validada conforme a legislação e metodologia de faturamento aplicáveis.

9. Histórico e auditoria

O sistema deverá preservar o histórico das alterações relevantes, principalmente:

- alterações de tarifas;
- alterações de tributação;
- alterações de bandeiras;
- alterações cadastrais importantes;
- registros de geração e pagamento de faturas.

Para alterações tarifárias, deverá ser possível identificar:

- quem realizou a operação;
- quando realizou;
- qual era o valor anterior;
- qual passou a ser o novo valor.

A decisão técnica entre campos de auditoria e tabelas específicas de histórico ainda deverá ser concluída durante a implementação. A preservação de histórico tarifário e de snapshots de fatura é regra aprovada.

10. Segurança e autorização

10.1 Autenticação

- Login com credenciais.
- Senhas armazenadas com hash seguro.
- Utilização de `PasswordEncoder`.
- BCrypt como implementação inicial recomendada.

10.2 Autorização

O sistema deverá distinguir `ADMINISTRADOR` e `CLIENTE`.

- Operações administrativas exigirão perfil `ADMINISTRADOR`.
- O `CLIENTE` somente poderá consultar dados associados à própria identidade.
- A proteção não deverá existir apenas nos endpoints.
- Os services também deverão validar a propriedade dos recursos quando necessário.

Exemplo: um cliente não poderá consultar uma fatura pertencente a outro cliente apenas informando seu UUID.

11. Banco de dados e migrations

11.1 Bancos

- H2 para desenvolvimento e testes, quando adequado.
- MySQL para produção.

11.2 Flyway

Flyway foi definido como mecanismo de controle das alterações estruturais do banco. As migrations deverão ser versionadas.

Exemplos ilustrativos:

```
 V1__criacao_tabelas_iniciais.sql
V2__criacao_unidade_consumidora.sql
V3__remodelagem_tarifas.sql
```

A nomenclatura definitiva dependerá do estado inicial do schema.

11.3 Hibernate

- Em produção, utilizar `ddl-auto=validate`.
- A estratégia de desenvolvimento deverá ser revisada para não conflitar com Flyway.
- `create-drop` não deverá permanecer como mecanismo concorrente de gerenciamento do schema quando as migrations estiverem controlando a estrutura.

11.4 Integridade

Deverão existir constraints adequadas para:

- chaves primárias;
- chaves estrangeiras;
- unicidade de faturas;
- identificação de medidores;
- unicidade de unidade por distribuidora e código;
- unicidade de leitura por medidor e instante;
- valores não negativos quando aplicável;
- integridade dos períodos de vigência.

A prevenção de sobreposição de vigências deverá ser garantida pela lógica transacional e, quando possível, por mecanismos adicionais do banco.

12. Arquitetura e organização das camadas

12.1 Entities

Representam o domínio e seus relacionamentos persistentes. Não deverão conter lógica de apresentação.

12.2 DTOs

Definem os contratos de entrada, atualização e resposta. Entidades JPA não deverão ser expostas diretamente nos endpoints.

12.3 Mappers

MapStruct será utilizado para conversão entre entidades e DTOs. Mappers não deverão assumir responsabilidades de negócio.

12.4 Repositories

Responsáveis pelo acesso aos dados. Consultas específicas deverão ficar nos repositories, evitando lógica de persistência espalhada pelos services.

12.5 Services

Responsáveis pelas regras de negócio, incluindo:

- cálculo do consumo;
- seleção da tarifa vigente;
- validação de períodos;
- aplicação de bandeiras;
- cálculo e registro de tributos;
- geração de faturas;
- registro de pagamentos;
- controle de acesso aos recursos.

Operações que exigem consistência deverão utilizar transações.

12.6 Controllers

Responsáveis pela exposição dos endpoints REST:

- receber requisições;
- validar entradas;
- acionar services;
- retornar respostas e códigos HTTP adequados.

12.7 Tratamento de exceções

A API deverá retornar respostas consistentes para:

- recurso não encontrado;
- dados inválidos;
- conflitos de negócio;
- falhas de autenticação;
- falhas de autorização;
- erros inesperados.

13. Enums consolidados

13.1 Enums presentes no modelo

Enum	Valores
`AbrangenciaTributaria`	`FEDERAL`, `ESTADUAL`, `MUNICIPAL`
`ClasseConsumo`	`RESIDENCIAL`, `INDUSTRIAL`, `COMERCIAL_SERVICOS`, `RURAL`, `PODER_PUBLICO`, `ILUMINACAO_PUBLICA`, `SERVICO_PUBLICO`, `CONSUMO_PROPRIO`
`ModalidadeTarifaria`	`CONVENCIONAL`, `BRANCA`
`OrigemConsumo`	`LEITURA`, `MANUAL`
`StatusFatura`	`PENDENTE`, `PAGO`
`SubgrupoTarifario`	`B1`, `B2`, `B3`, `B4`

`TipoBandeira`	`VERDE`, `AMARELA`, `VERMELHA_PATAMAR_1`, `VERMELHA_PATAMAR_2`
`TipoCalculoTributario`	`PERCENTUAL`, `VALOR_FIXO`
`TipoLeitura`	`REAL`, `ESTIMADA`
`TipoLigacao`	`MONOFASICO`, `BIFASICO`, `TRIFASICO`
`TipoMedidor`	`ELETROMECANICO`, `ELETRONICO`, `INTELIGENTE`
`TipoTributo`	`ICMS`, `PIS`, `COFINS`, `CIP_COSIP`
`TipoUsuario`	`ADMINISTRADOR`, `CLIENTE`

14. Integração com ANEEL e fontes oficiais

A integração automática com dados oficiais não faz parte da primeira implementação.

Inicialmente, o cadastro será manual e administrativo. A arquitetura deverá permitir futuramente:

- importação de tarifas homologadas;
- atualização de TE e TUSD;
- importação de informações oficiais de bandeiras;
- atualização de dados regulatórios;
- rastreabilidade da fonte dos dados.

A implementação futura deverá utilizar fontes oficiais e preservar o histórico dos dados importados.

15. Regras de negócio consolidadas

Código	Regra
RN-01	Valores de consumo, tarifas e tributos não poderão ser negativos quando aplicável.
RN-02	Um cliente poderá possuir várias unidades consumidoras.
RN-03	Uma unidade consumidora pertence a um único cliente.
RN-04	Um medidor pertence a uma unidade consumidora.
RN-05	Uma unidade poderá possuir histórico de vários medidores.
RN-06	O histórico de leituras deverá ser preservado na substituição do medidor.
RN-07	O consumo será calculado pela diferença entre leituras consecutivas do mesmo medidor, observadas as situações operacionais.
RN-08	A fatura pertence à unidade consumidora.
RN-09	Não poderá existir mais de uma fatura por unidade e competência.
RN-10	A competência será representada pelo primeiro dia do mês.
RN-11	A tarifa deverá ser válida no início da competência.
RN-12	Não poderão existir vigências tarifárias sobrepostas para a mesma combinação tarifária.
RN-13	Um reajuste deverá encerrar o registro anterior e criar outro.
RN-14	Bandeiras serão independentes da tarifa TE/TUSD.
RN-15	Tributos serão modelados separadamente.
RN-16	A fatura deverá preservar snapshots dos valores aplicados.
RN-17	Uma fatura nova terá status `PENDENTE`.
RN-18	O pagamento registrado altera o status para `PAGO`.
RN-19	Pagamento duplicado não será permitido.
RN-20	O cliente somente poderá acessar seus próprios dados.
RN-21	Operações administrativas exigirão perfil `ADMINISTRADOR`.
RN-22	Valores monetários utilizarão `BigDecimal`.
RN-23	Histórico tarifário e tributário deverá ser preservado.
RN-24	Migrations serão gerenciadas pelo Flyway.
RN-25	Alterações relevantes deverão possuir rastreabilidade.
RN-26	O valor da bandeira e seu acionamento oficial serão registros distintos.
RN-27	Um ciclo de faturamento poderá conter mais de um período de bandeira; a aplicação deverá considerar os períodos abrangidos.
RN-28	A fatura deverá identificar a origem do consumo (`LEITURA` ou `MANUAL`).

RN-29	Uma unidade consumidora poderá ser desativada sem apagar seu histórico.
RN-30	Um cliente poderá existir sem usuário; usuário `CLIENTE` exige associação única com um cliente.

16. Constraints e índices definidos no modelo

Esta seção descreve o que foi encontrado nas entidades JPA inspecionadas em 10/11/2026. A efetiva criação no banco depende do schema/migrations e deverá ser validada por testes de persistência.

16.1 Unicidade e índices encontrados

- `UnidadeConsumidora`: unique `(distribuidora, codigo)` e índice `idx_unidade_cliente`.
- `Medidor`: `numero` globalmente único e índice `idx_medidor_unidade_ativo`.
- `Leitura`: unique `(medidor_id, data_leitura)`.
- `Fatura`: unique `(unidade_consumidora_id, competencia)` e índices para competência e status.
- `Usuario`: username único e associação `cliente_id` única pelo `@OneToOne`.
- `Taxa`: índice composto para distribuidora, subgrupo, classe, modalidade e início de vigência.
- `BandeiraTarifaria`: índice por tipo de bandeira e início de vigência.
- `AcionamentoBandeira`: índice por início e fim de vigência.
- `FaturaBandeiraAplicada`: índices para `fatura_id` e `acionamento_bandeira_id`.
- `RegraTributaria`: índice de busca por tipo, abrangência, estado, código IBGE e início de vigência.
- `FaturaTributo`: índices para fatura, tipo de tributo e regra tributária.

16.2 Checks encontrados

- `Taxa`: TE e TUSD não negativas; fim de vigência nulo ou posterior ao início.
- `BandeiraTarifaria`: adicional não negativo; fim de vigência nulo ou posterior ao início.
- `AcionamentoBandeira`: fim de vigência nulo ou posterior ao início.
- `Leitura`: valor não negativo.
- `Fatura`: consumo não negativo.
- `FaturaBandeiraAplicada`: adicional por kWh, consumo considerado e valor adicional não negativos; fim da aplicação igual ou posterior ao início.
- `RegraTributaria`: alíquota e valor fixo nulos ou não negativos; fim de vigência nulo ou posterior ao início.
- `Usuario`: domínio permitido para o tipo de usuário.

16.3 Complementações ainda necessárias

- A entidade `Fatura` inspecionada possui check para consumo, mas não foram encontrados checks para todos os campos monetários.
- A coerência entre `tipoCalculo` e os campos `aliquota` / `valorFixo` de `RegraTributaria` ainda deverá ser garantida pelo service e, se adequado, por constraint adicional.
- A ausência de sobreposição tarifária é regra transacional de service; um índice comum não garante essa regra.
- A regra de no máximo um medidor ativo por unidade deverá ser aplicada no service e, se suportado pelo banco utilizado, reforçada por mecanismo de integridade adequado.
- Testes deverão confirmar se todas as constraints anotadas são aceitas e criadas tanto no H2 quanto no MySQL.

17. Diagnóstico do código inspecionado em 10/11/2026

A inspeção foi realizada sobre o ZIP `powermanager.zip` e arquivos do repositório GitHub `GustavoJ-dev/PowerManager`. A análise distingue a modelagem de domínio já atualizada das camadas que ainda refletem o modelo anterior.

17.1 Encontrado nas entidades e enums

As entidades remodeladas abaixo foram encontradas no código:

- `Cliente`;
- `UnidadeConsumidora`;
- `Medidor`;
- `Leitura`;
- `Taxa`;
- `BandeiraTarifaria`;
- `AcionamentoBandeira`;
- `RegraTributaria`;
- `Fatura`;
- `FaturaBandeiraAplicada`;
- `FaturaTributo`;
- `Usuario`.

Os enums consolidados também foram encontrados.

17.2 Camadas que precisam ser alinhadas

No ZIP inspecionado, DTOs, mappers, repositories e services ainda estão presentes somente para parte do modelo anterior (Cliente, Medidor, Taxa, Fatura e Usuario). Não foram encontrados, nesse conjunto de arquivos, fluxos completos específicos para `UnidadeConsumidora`, `Leitura`, `BandeiraTarifaria`, `AcionamentoBandeira`, `RegraTributaria`, `FaturaBandeiraAplicada` e `FaturaTributo`.

A documentação antiga do README ainda descreve o modelo anterior — cliente diretamente associado a medidor, tarifa genérica com custo por unidade, aluguel e taxas adicionais. Essa descrição deve ser substituída pelo modelo atual.

17.3 Segurança

O modelo de `Usuario` possui `passwordHash`, mas o nome do campo não comprova que a senha esteja efetivamente protegida. A implementação deverá usar `PasswordEncoder` e Spring Security. Não considerar autenticação pronta sem validar o fluxo completo.

17.4 Banco e build

O repositório contém configuração de H2 para `dev` e MySQL para `prod`; o profile `prod` utiliza `ddl-auto: validate`. O profile `dev` atualmente usa `create-drop`.

Flyway está definido como estratégia do projeto, mas sua configuração, dependências e migrations devem ser confirmadas antes de marcar essa etapa como concluída.

O ZIP de código disponibilizado para esta revisão não incluiu `pom.xml`, recursos de configuração, migrations, controllers ou testes. Por isso, não foi possível comprovar por esse ZIP uma compilação completa nem a execução dos testes de integração das entidades. A confirmação final depende de rodar a suíte no repositório completo.

18. Plano de refatoração

A sequência aprovada para a refatoração é:

1. Revisão da arquitetura — concluída como diagnóstico inicial.
2. Remodelagem das entidades — realizada no código; falta concluir a validação de persistência real no H2.
3. Consolidação dos enums — enums atuais encontrados no código; validar compatibilidade e testes.
4. Remodelagem dos DTOs.
5. Atualização dos mappers.
6. Remodelagem dos repositories.
7. Implementação dos services e regras de negócio.
8. Implementação dos controllers e endpoints.
9. Implementação de Spring Security e autorização.
10. Configuração do Flyway e migrations.
11. Testes unitários, de integração e persistência.
12. Validação final de ponta a ponta.

Regra de execução: não avançar para a etapa seguinte sem validar a etapa atual, conforme o fluxo combinado. Alterações relacionadas deverão ser entregues em conjunto para evitar conflitos de código.

19. Pontos ainda pendentes de definição técnica ou regulatória

Item	Pendência
Tributação	Fórmulas, bases e hipóteses de incidência de cada tributo.
CIP/COSIP	Estrutura e critérios de cálculo municipais.
Bandeiras	Distribuição do consumo por períodos no ciclo, conforme regra regulatória aplicável.
Leituras	Tratamento de rollover, troca de medidor e leituras estimadas.
Consumo mínimo	Aplicação conforme condições regulatórias e técnicas.
Distribuidora	Manter `String` inicialmente; entidade própria é evolução futura.
Endereço	Possível normalização e identificação oficial do município.
Auditoria	Campos de auditoria ou tabelas específicas de histórico.
Modalidade branca	Regras horárias e faixas de consumo, fora da primeira implementação.
Segurança	Configuração completa de autenticação, autorização e propriedade dos recursos.
Persistência	Testes reais de criação, relacionamento, leitura e constraints no H2/MySQL.
Migrations	Estado inicial do schema e sequência definitiva de migrations Flyway.

20. Critérios para considerar a refatoração concluída

A refatoração não estará concluída apenas porque as entidades compilam. Será necessário:

- entidades e relacionamentos validados por testes de persistência;
- enums coerentes com os atributos do domínio;
- DTOs atualizados e sem exposição de entidades;
- mappers compatíveis com os novos relacionamentos;
- repositories com consultas adequadas;
- services implementando as regras aprovadas;
- transações nas operações críticas;
- controllers com validação e respostas HTTP coerentes;
- Spring Security com hash seguro e autorização por perfil e propriedade;
- Flyway gerenciando o schema;
- testes unitários e de integração passando;
- snapshots de fatura preservando valores históricos;
- cálculos tributários e de bandeiras validados conforme regras aplicáveis;
- ausência de regressões nos fluxos anteriores ainda necessários.

21. Estado consolidado

As regras de negócio fundamentais foram discutidas e aprovadas. A remodelagem das entidades e dos enums foi aplicada ao código disponibilizado, incluindo unidade consumidora, leituras, tarifa TE/TUSD, bandeiras, tributação e snapshots de fatura.

A refatoração das camadas de aplicação ainda precisa acompanhar o novo domínio. DTOs, mappers, repositories, services, controllers, segurança, migrations e testes de integração deverão ser alinhados antes de considerar o backend pronto.

Este documento é a referência consolidada para a continuidade da refatoração. Alterações futuras nas regras deverão ser discutidas e aprovadas antes de serem incorporadas.